

Competitive Programming

Section 69 Editorial: Master+ Structures And Trees Mixed Contest

Competitive Programming Course

Table of contents

1	A. Snapshot Range Rank	3
2	B. Seasonal Component Size	7
3	C. Route Matrices	11
4	D. Beacon Distance Sum	16
5	E. Weighted Marked Pairs	21
6	F. Subtree Mode Profile	25
7	Stress testing and carry-forward	28

1 A. Snapshot Range Rank

Every append creates a node in the version tree and a persistent value-frequency root. Binary lifting finds the ancestor version whose sequence length is r and the ancestor of length $l-1$. Subtracting their segment counts leaves exactly positions $[l, r]$; k -th descent on the two roots returns the rank.

Proof. A version's ancestors are precisely its sequence prefixes because each version appends once to its parent. Persistent-root differences therefore remove exactly the first $l-1$ values from the first r . At every k -th descent, the difference of left counts is the number of remaining range values in the left half, so rank selection is correct.

Time is $O(\log q + \log M)$ per query or append; memory is $O(q(\log q + \log M))$. Test branching versions and intervals not ending at the queried version depth.

```
#include <bits/stdc++.h>
using namespace std;
struct Node {
    int left = 0, right = 0, count = 0;
};
vector<Node> tree(1);
int add(int old, int lo, int hi, int pos) {
    int node = tree.size();
    tree.push_back(tree[old]);
    tree[node].count++;
    if (lo != hi) {
        int mid = (lo + hi) / 2;
        if (pos <= mid)
            tree[node].left = add(tree[old].left, lo, mid, pos);
        else
            tree[node].right = add(tree[old].right, mid + 1, hi, pos);
    }
    return node;
}
int kth(int before, int after, int lo, int hi, int k) {
    if (lo == hi) return lo;
    int amount = tree[tree[after].left].count - tree[tree[before].left].count,
        mid = (lo + hi) / 2;
    if (k <= amount) return kth(tree[before].left, tree[after].left, lo, mid, k);
    return kth(tree[before].right, tree[after].right, mid + 1, hi, k - amount);
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int maximum, q;
    if (!(cin >> maximum >> q)) return 0;
```

```

int log = 20;
while ((1 << log) <= q) log++;
vector<vector<int>> up(log, vector<int>(q + 1));
vector<int> root(q + 1), depth(q + 1);
int versions = 0;
auto ancestor = [&](int v, int steps) {
    for (int j = 0; j < log; j++)
        if (steps >> j & 1) v = up[j][v];
    return v;
};
while (q--) {
    char type;
    cin >> type;
    if (type == 'A') {
        int v, x;
        cin >> v >> x;
        ++versions;
        depth[versions] = depth[v] + 1;
        up[0][versions] = v;
        for (int j = 1; j < log; j++) up[j][versions] = up[j - 1][up[j - 1][versions]];
        root[versions] = add(root[v], 1, maximum, x);
    } else {
        int v, l, r, k;
        cin >> v >> l >> r >> k;
        int right_version = ancestor(v, depth[v] - r),
            left_version = ancestor(v, depth[v] - (l - 1));
        cout << kth(root[left_version], root[right_version], 1, maximum, k) << '\n';
    }
}
}
}

```

```

import sys

def main():
    tokens = sys.stdin.buffer.read().split()
    if not tokens:
        return
    idx = 0
    maximum = int(tokens[idx])
    q = int(tokens[idx + 1])
    idx += 2
    log = (q + 1).bit_length()
    up = [[0] * (q + 1) for _ in range(log)]
    roots = [0] * (q + 1)
    depth = [0] * (q + 1)
    left = [0]
    right = [0]
    count = [0]
    versions = 0

```

```

def add(old, lo, hi, pos):
    node = len(count)
    left.append(left[old])
    right.append(right[old])
    count.append(count[old] + 1)
    if lo != hi:
        mid = (lo + hi) // 2
        if pos <= mid:
            left[node] = add(left[old], lo, mid, pos)
        else:
            right[node] = add(right[old], mid + 1, hi, pos)
    return node

def ancestor(v, steps):
    bit = 0
    while steps:
        if steps & 1:
            v = up[bit][v]
        steps //= 2
        bit += 1
    return v

def kth(before, after, k):
    lo, hi = 1, maximum
    while lo != hi:
        amount = count[left[after]] - count[left[before]]
        mid = (lo + hi) // 2
        if k <= amount:
            before = left[before]
            after = left[after]
            hi = mid
        else:
            k -= amount
            before = right[before]
            after = right[after]
            lo = mid + 1
    return lo

out = []
for _ in range(q):
    kind = tokens[idx].decode()
    idx += 1
    if kind == "A":
        v = int(tokens[idx])
        x = int(tokens[idx + 1])
        idx += 2
        versions += 1
        depth[versions] = depth[v] + 1
        up[0][versions] = v
        for j in range(1, log):
            up[j][versions] = up[j - 1][up[j - 1][versions]]

```

```

        roots[versions] = add(roots[v], 1, maximum, x)
    else:
        v, l, r, k = map(int, tokens[idx : idx + 4])
        idx += 4
        rv = ancestor(v, depth[v] - r)
        lv = ancestor(v, depth[v] - l + 1)
        out.append(str(kth(roots[lv], roots[rv], k)))
print("\n".join(out))

if __name__ == "__main__":
    main()

```

2 B. Seasonal Component Size

Convert every edge lifetime to a half-open interval, store it in a time segment tree, and DFS with rollback DSU. At a size-query leaf, return the size stored at the representative of u .

Proof. Exactly the active edges cover the leaf's time, so its ancestor time-tree nodes unite exactly the current graph. Union-by-size DSU stores the cardinality of each resulting component. Snapshot rollback prevents one time subtree from affecting its sibling.

Total time is $O(q \log q \log n)$ and memory $O(q \log q + n)$. Test a component splitting after a bridge removal and edges active through the final operation.

```
#include <bits/stdc++.h>
using namespace std;
struct DSU {
    vector<int> p, s;
    vector<pair<int, int>> h;
    DSU(int n) : p(n), s(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) {
        while (x != p[x]) x = p[x];
        return x;
    }
    int snap() { return h.size(); }
    void join(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return;
        if (s[a] < s[b]) swap(a, b);
        h.push_back({b, s[a]});
        p[b] = a;
        s[a] += s[b];
    }
    void rollback(int z) {
        while ((int)h.size() > z) {
            auto [b, old] = h.back();
            h.pop_back();
            int a = p[b];
            s[a] = old;
            p[b] = b;
        }
    }
    int component(int x) { return s[find(x)]; }
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

int n, q;
if (!(cin >> n >> q)) return 0;
struct Op {
    char type;
    int u, v;
};
vector<Op> ops(q);
map<pair<int, int>, int> open;
vector<vector<pair<int, int>>> seg(4 * q + 4);
function<void(int, int, int, int, int, pair<int, int>>> add =
    [&](int x, int l, int r, int ql, int qr, pair<int, int> e) {
        if (ql >= r || qr <= l) return;
        if (ql <= l && r <= qr) {
            seg[x].push_back(e);
            return;
        }
        int m = (l + r) / 2;
        add(2 * x, l, m, ql, qr, e);
        add(2 * x + 1, m, r, ql, qr, e);
    });
for (int t = 0; t < q; t++) {
    cin >> ops[t].type >> ops[t].u;
    --ops[t].u;
    ops[t].v = -1;
    if (ops[t].type != 'S') {
        cin >> ops[t].v;
        --ops[t].v;
        if (ops[t].u > ops[t].v) swap(ops[t].u, ops[t].v);
        auto e = pair{ops[t].u, ops[t].v};
        if (ops[t].type == '+')
            open[e] = t;
        else
            add(1, 0, q, open[e], t, e), open.erase(e);
    }
}
for (auto [e, t] : open) add(1, 0, q, t, q, e);
DSU dsu(n);
function<void(int, int, int)> dfs = [&](int x, int l, int r) {
    int z = dsu.snap();
    for (auto [u, v] : seg[x]) dsu.join(u, v);
    if (r - l == 1) {
        if (ops[l].type == 'S') cout << dsu.component(ops[l].u) << '\n';
    } else {
        int m = (l + r) / 2;
        dfs(2 * x, l, m);
        dfs(2 * x + 1, m, r);
    }
    dsu.rollback(z);
};
if (q) dfs(1, 0, q);
}

```

```

import sys

def main():
    tokens = sys.stdin.buffer.read().split()
    if not tokens:
        return
    idx = 0
    n = int(tokens[idx])
    q = int(tokens[idx + 1])
    idx += 2
    ops = []
    opened = {}
    seg = [[] for _ in range(4 * q + 4)]

    def add(x, l, r, ql, qr, e):
        if ql >= r or qr <= l:
            return
        if ql <= l and r <= qr:
            seg[x].append(e)
            return
        m = (l + r) // 2
        add(2 * x, l, m, ql, qr, e)
        add(2 * x + 1, m, r, ql, qr, e)

    for time in range(q):
        kind = tokens[idx].decode()
        u = int(tokens[idx + 1]) - 1
        idx += 2
        v = -1
        if kind != "S":
            v = int(tokens[idx]) - 1
            idx += 1
            if u > v:
                u, v = v, u
            if kind == "+":
                opened[(u, v)] = time
            else:
                add(1, 0, q, opened.pop((u, v)), time, (u, v))
        ops.append((kind, u, v))
    for edge, start in opened.items():
        add(1, 0, q, start, q, edge)
    parent = list(range(n))
    size = [1] * n
    history = []

    def find(x):
        while x != parent[x]:
            x = parent[x]
        return x

```

```

def join(a, b):
    a = find(a)
    b = find(b)
    if a == b:
        return
    if size[a] < size[b]:
        a, b = b, a
    history.append((b, size[a]))
    parent[b] = a
    size[a] += size[b]

def dfs(x, l, r):
    snap = len(history)
    for u, v in seg[x]:
        join(u, v)
    if r - l == 1:
        if ops[l][0] == "S":
            output.append(str(size[find(ops[l][1])]))
    else:
        m = (l + r) // 2
        dfs(2 * x, l, m)
        dfs(2 * x + 1, m, r)
    while len(history) > snap:
        b, old = history.pop()
        a = parent[b]
        size[a] = old
        parent[b] = b

output = []
if q:
    dfs(1, 0, q)
print("\n".join(output))

if __name__ == "__main__":
    main()

```

3 C. Route Matrices

Heavy-light decomposition gives $O(\log n)$ intervals. Every segment node stores left-to-right and right-to-left matrix products. Accumulate u-side chunks in travel order, prepend v-side chunks, and place the final same-chain interval between them.

Proof. Segment induction proves both directional products. HLD chunks are a disjoint partition of the path, and the two accumulators concatenate them in the exact vertex order. Associativity permits chunking; no commutativity is assumed. Thus the printed matrix is the requested product.

Updates are $O(\log n)$, queries $O(\log^2 n)$, memory $O(n)$. Use matrices where $AB \neq BA$ when testing reverse paths.

```
#include <bits/stdc++.h>
using namespace std;
constexpr long long MOD = 998244353;
struct Matrix {
    long long a = 1, b = 0, c = 0, d = 1;
};
Matrix multiply(Matrix x, Matrix y) {
    return {(x.a * y.a + x.b * y.c) % MOD, (x.a * y.b + x.b * y.d) % MOD,
            (x.c * y.a + x.d * y.c) % MOD, (x.c * y.b + x.d * y.d) % MOD};
}
struct Item {
    Matrix forward, backward;
};
Item merge(Item x, Item y) {
    return {multiply(x.forward, y.forward), multiply(y.backward, x.backward)};
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    vector<Matrix> value(n);
    for (auto& m : value) cin >> m.a >> m.b >> m.c >> m.d;
    vector<vector<int>> g(n);
    for (int i = 1, u, v; i < n; i++) {
        cin >> u >> v;
        --u;
        --v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<int> parent(n, -1), depth(n), order{0};
```

```

for (int i = 0; i < (int)order.size(); i++) {
    int u = order[i];
    for (int v : g[u])
        if (v != parent[u]) parent[v] = u, depth[v] = depth[u] + 1, order.push_back(v);
}
vector<int> size(n, 1), heavy(n, -1);
for (int i = n - 1; i; i--) {
    int u = order[i], p = parent[u];
    size[p] += size[u];
    if (heavy[p] < 0 || size[u] > size[heavy[p]]) heavy[p] = u;
}
vector<int> head(n), pos(n);
int timer = 0;
vector<pair<int, int>> todo{{0, 0}};
while (!todo.empty()) {
    auto [u, h] = todo.back();
    todo.pop_back();
    for (; u != -1; u = heavy[u]) {
        head[u] = h;
        pos[u] = timer++;
        for (int v : g[u])
            if (parent[v] == u && v != heavy[u]) todo.push_back({v, v});
    }
}
int base = 1;
while (base < n) base *= 2;
vector<Item> tree(2 * base);
for (int u = 0; u < n; u++) tree[base + pos[u]] = {value[u], value[u]};
for (int p = base - 1; p; p--) tree[p] = merge(tree[2 * p], tree[2 * p + 1]);
auto range = [&](int l, int r) {
    Item x{}, y{};
    for (l += base, r += base + 1; l < r; l /= 2, r /= 2) {
        if (l & 1) x = merge(x, tree[l++]);
        if (r & 1) y = merge(tree[--r], y);
    }
    return merge(x, y);
};
while (q--) {
    char type;
    int u;
    cin >> type >> u;
    --u;
    if (type == 'U') {
        Matrix m;
        cin >> m.a >> m.b >> m.c >> m.d;
        int p = base + pos[u];
        tree[p] = {m, m};
        for (p /= 2; p; p /= 2) tree[p] = merge(tree[2 * p], tree[2 * p + 1]);
    } else {
        int v;
        cin >> v;
    }
}

```

```

--v;
Matrix left{}, right{};
while (head[u] != head[v]) {
    if (depth[head[u]] >= depth[head[v]])
        left = multiply(left, range(pos[head[u]], pos[u]).backward),
        u = parent[head[u]];
    else
        right = multiply(range(pos[head[v]], pos[v]).forward, right),
        v = parent[head[v]];
}
Matrix middle = pos[u] <= pos[v] ? range(pos[u], pos[v]).forward
                                : range(pos[v], pos[u]).backward;
Matrix answer = multiply(multiply(left, middle), right);
cout << answer.a << ' ' << answer.b << ' ' << answer.c << ' ' << answer.d << '\n';
}
}
}

```

```

import sys

MOD = 998_244_353
IDENTITY = (1, 0, 0, 1)

def mul(x, y):
    return (
        (x[0] * y[0] + x[1] * y[2]) % MOD,
        (x[0] * y[1] + x[1] * y[3]) % MOD,
        (x[2] * y[0] + x[3] * y[2]) % MOD,
        (x[2] * y[1] + x[3] * y[3]) % MOD,
    )

def merge(x, y):
    return mul(x[0], y[0]), mul(y[1], x[1])

def main():
    t = sys.stdin.buffer.read().split()
    if not t:
        return
    z = 0
    n = int(t[z])
    q = int(t[z + 1])
    z += 2
    values = []
    for _ in range(n):
        values.append(tuple(map(int, t[z : z + 4])))
        z += 4
    g = [[] for _ in range(n)]
    for _ in range(n - 1):

```

```

    u = int(t[z]) - 1
    v = int(t[z + 1]) - 1
    z += 2
    g[u].append(v)
    g[v].append(u)
parent = [-1] * n
depth = [0] * n
order = [0]
for u in order:
    for v in g[u]:
        if v != parent[u]:
            parent[v] = u
            depth[v] = depth[u] + 1
            order.append(v)
size = [1] * n
heavy = [-1] * n
for u in reversed(order[1:]):
    p = parent[u]
    size[p] += size[u]
    heavy[p] = u if heavy[p] < 0 or size[u] > size[heavy[p]] else heavy[p]
head = [0] * n
pos = [0] * n
timer = 0
todo = [(0, 0)]
while todo:
    u, h = todo.pop()
    while u != -1:
        head[u] = h
        pos[u] = timer
        timer += 1
        for v in g[u]:
            if parent[v] == u and v != heavy[u]:
                todo.append((v, v))
        u = heavy[u]
base = 1
while base < n:
    base *= 2
tree = [(IDENTITY, IDENTITY) for _ in range(2 * base)]
for u, m in enumerate(values):
    tree[base + pos[u]] = (m, m)
for p in range(base - 1, 0, -1):
    tree[p] = merge(tree[2 * p], tree[2 * p + 1])

def query(l, r):
    x = y = (IDENTITY, IDENTITY)
    l += base
    r += base + 1
    while l < r:
        if l & 1:
            x = merge(x, tree[l])
        l += 1

```

```

        if r & 1:
            r -= 1
            y = merge(tree[r], y)
        l //= 2
        r //= 2
    return merge(x, y)

out = []
for _ in range(q):
    kind = t[z].decode()
    u = int(t[z + 1]) - 1
    z += 2
    if kind == "U":
        m = tuple(map(int, t[z : z + 4]))
        z += 4
        p = base + pos[u]
        tree[p] = (m, m)
        p //= 2
        while p:
            tree[p] = merge(tree[2 * p], tree[2 * p + 1])
            p //= 2
    else:
        v = int(t[z]) - 1
        z += 1
        left = right = IDENTITY
        while head[u] != head[v]:
            if depth[head[u]] >= depth[head[v]]:
                left = mul(left, query(pos[head[u]], pos[u])[1])
                u = parent[head[u]]
            else:
                right = mul(query(pos[head[v]], pos[v])[0], right)
                v = parent[head[v]]
        middle = (
            query(pos[u], pos[v])[0]
            if pos[u] <= pos[v]
            else query(pos[v], pos[u])[1]
        )
        out.append(" ".join(map(str, mul(mul(left, middle), right))))
print("\n".join(out))

if __name__ == "__main__":
    main()

```

4 D. Beacon Distance Sum

For each centroid c , maintain the active count and sum of active distances to c . Also maintain the same pair for each component branch incident to c . For query vertex u , each centroid ancestor contributes

```
total_sum[c] + total_count[c] * dist(u,c),
```

then subtracts the corresponding branch subtotal when u lies in a branch.

Proof. At centroid c , the total expression counts routes through c to all active vertices. Those in u 's same branch may not pass through c and will be accounted for at a lower centroid, so subtract exactly that branch. For each active vertex, the first centroid separating it from u is the unique level where it is included but not subtracted; there the route through the centroid is its true path. Hence every distance appears once.

Build uses $O(n \log n)$ memory/time. Toggle and query are $O(\log n)$ expected with hash-table branch totals. Test empty/all-active states and repeated toggles.

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
struct Entry {
    int centroid, distance, branch;
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    vector<vector<int>>> g(n);
    for (int i = 1, u, v; i < n; i++) {
        cin >> u >> v;
        --u;
        --v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<char> blocked(n), active(n);
    vector<int> subtree(n);
    vector<vector<Entry>>> path(n);
    vector<int> todo{0};
    while (!todo.empty()) {
        int entry = todo.back();
        todo.pop_back();
        vector<int> nodes{entry}, parent(n, -2);
```

```

parent[entry] = -1;
for (int i = 0; i < (int)nodes.size(); i++) {
    int u = nodes[i];
    for (int v : g[u])
        if (!blocked[v] && parent[v] == -2) parent[v] = u, nodes.push_back(v);
}
for (int i = nodes.size() - 1; i >= 0; i--) {
    int u = nodes[i];
    subtree[u] = 1;
    for (int v : g[u])
        if (!blocked[v] && parent[v] == u) subtree[u] += subtree[v];
}
int total = nodes.size(), centroid = entry;
for (int u : nodes) {
    int largest = total - subtree[u];
    for (int v : g[u])
        if (!blocked[v] && parent[v] == u) largest = max(largest, subtree[v]);
    if (largest * 2 <= total) {
        centroid = u;
        break;
    }
}
path[centroid].push_back({centroid, 0, -1});
for (int first : g[centroid])
    if (!blocked[first]) {
        vector<tuple<int, int, int>> stack{{first, centroid, 1}};
        while (!stack.empty()) {
            auto [u, p, d] = stack.back();
            stack.pop_back();
            path[u].push_back({centroid, d, first});
            for (int v : g[u])
                if (v != p && !blocked[v]) stack.push_back({v, u, d + 1});
        }
    }
blocked[centroid] = 1;
for (int v : g[centroid])
    if (!blocked[v]) todo.push_back(v);
}
vector<int64> count(n), sum(n);
unordered_map<long long, pair<int64, int64>> excluded;
auto key = [&](int c, int b) { return 1LL * c * n + b; };
while (q--) {
    char type;
    int u;
    cin >> type >> u;
    --u;
    if (type == 'T') {
        int delta = active[u] ? -1 : 1;
        active[u] ^= 1;
        for (auto [c, d, b] : path[u]) {
            count[c] += delta;

```

```

        sum[c] += 1LL * delta * d;
        if (b != -1) {
            auto& state = excluded[key(c, b)];
            state.first += delta;
            state.second += 1LL * delta * d;
        }
    }
} else {
    int64 answer = 0;
    for (auto [c, d, b] : path[u]) {
        answer += sum[c] + count[c] * d;
        if (b != -1) {
            auto state = excluded[key(c, b)];
            answer -= state.second + state.first * d;
        }
    }
    cout << answer << '\n';
}
}
}
}

```

```

import sys

def main():
    t = sys.stdin.buffer.read().split()
    if not t:
        return
    z = 0
    n = int(t[z])
    q = int(t[z + 1])
    z += 2
    g = [[] for _ in range(n)]
    for _ in range(n - 1):
        u = int(t[z]) - 1
        v = int(t[z + 1]) - 1
        z += 2
        g[u].append(v)
        g[v].append(u)
    blocked = [False] * n
    subtree = [0] * n
    paths = [[] for _ in range(n)]
    todo = [0]
    while todo:
        entry = todo.pop()
        nodes = [entry]
        parent = {entry: -1}
        for u in nodes:
            for v in g[u]:
                if not blocked[v] and v not in parent:
                    parent[v] = u

```

```

        nodes.append(v)
    for u in reversed(nodes):
        subtree[u] = 1 + sum(subtree[v] for v in g[u] if parent.get(v) == u)
    total = len(nodes)
    centroid = entry
    for u in nodes:
        largest = total - subtree[u]
        for v in g[u]:
            if not blocked[v] and parent.get(v) == u:
                largest = max(largest, subtree[v])
        if largest * 2 <= total:
            centroid = u
            break
    paths[centroid].append((centroid, 0, -1))
    for first in g[centroid]:
        if not blocked[first]:
            stack = [(first, centroid, 1)]
            while stack:
                u, p, d = stack.pop()
                paths[u].append((centroid, d, first))
                for v in g[u]:
                    if v != p and not blocked[v]:
                        stack.append((v, u, d + 1))
    blocked[centroid] = True
    for v in g[centroid]:
        if not blocked[v]:
            todo.append(v)
    count = [0] * n
    distance_sum = [0] * n
    excluded = {}
    active = [False] * n
    out = []
    for _ in range(q):
        kind = t[z].decode()
        u = int(t[z + 1]) - 1
        z += 2
        if kind == "T":
            delta = -1 if active[u] else 1
            active[u] = not active[u]
            for c, d, b in paths[u]:
                count[c] += delta
                distance_sum[c] += delta * d
                if b != -1:
                    old_count, old_sum = excluded.get((c, b), (0, 0))
                    excluded[c, b] = (old_count + delta, old_sum + delta * d)
        else:
            answer = 0
            for c, d, b in paths[u]:
                answer += distance_sum[c] + count[c] * d
                if b != -1:
                    old_count, old_sum = excluded.get((c, b), (0, 0))

```

```
        answer -= old_sum + old_count * d
        out.append(str(answer))
    print("\n".join(out))

if __name__ == "__main__":
    main()
```

5 E. Weighted Marked Pairs

Construct the virtual tree of marked vertices and necessary LCAs. Let `sub[u]` be the sum of coefficients below virtual edge (p, u) , and `total` the sum of all query coefficients. Add

```
(dist[u]-dist[p]) * sub[u] * (total-sub[u]).
```

Proof. Expanding `sub*(total-sub)` selects every cross-edge marked pair with weight product $c_i \cdot c_j$ exactly once. Such a pair's original path uses every edge compressed into (p, u) . Summing virtual edges therefore gives each pair's coefficient product times its complete distance.

After $O(n \log n)$ LCA preprocessing, a query is $O(k \log k)$ and $O(k)$ memory.

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    vector<vector<pair<int, int>>> g(n);
    for (int i = 1, u, v, w; i < n; i++) {
        cin >> u >> v >> w;
        --u;
        --v;
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    int log = bit_width((unsigned)n);
    vector<vector<int>> up(log, vector<int>(n));
    vector<int> tin(n), tout(n);
    vector<int64> distance(n);
    int timer = 0;
    vector<tuple<int, int, int>> stack{{0, 0, 0}};
    while (!stack.empty()) {
        auto [u, p, state] = stack.back();
        stack.pop_back();
        if (!state) {
            tin[u] = timer++;
            up[0][u] = p;
            for (int j = 1; j < log; j++) up[j][u] = up[j - 1][up[j - 1][u]];
            stack.push_back({u, p, 1});
            for (auto it = g[u].rbegin(); it != g[u].rend(); ++it)
                if (it->first != p)
                    distance[it->first] = distance[u] + it->second,
```

```

        stack.push_back({it->first, u, 0});
    } else
        tout[u] = timer - 1;
}
auto ancestor = [&](int u, int v) { return tin[u] <= tin[v] && tin[v] <= tout[u]; };
auto lca = [&](int u, int v) {
    if (ancestor(u, v)) return u;
    if (ancestor(v, u)) return v;
    for (int j = log - 1; j >= 0; j--)
        if (!ancestor(up[j][u], v)) u = up[j][u];
    return up[0][u];
};
vector<int64> weight(n), subtree(n);
vector<int> parent(n);
while (q--) {
    int k;
    cin >> k;
    vector<int> nodes(k);
    int64 total = 0;
    for (int i = 0; i < k; i++) {
        cin >> nodes[i] >> weight[--nodes[i]];
        total += weight[nodes[i]];
    }
    sort(nodes.begin(), nodes.end(), [&](int a, int b) { return tin[a] < tin[b]; });
    for (int i = 1; i < k; i++) nodes.push_back(lca(nodes[i - 1], nodes[i]));
    sort(nodes.begin(), nodes.end(), [&](int a, int b) { return tin[a] < tin[b]; });
    nodes.erase(unique(nodes.begin(), nodes.end()), nodes.end());
    vector<int> st;
    for (int u : nodes) {
        while (!st.empty() && !ancestor(st.back(), u)) st.pop_back();
        parent[u] = st.empty() ? -1 : st.back();
        st.push_back(u);
        subtree[u] = weight[u];
    }
    int64 answer = 0;
    for (int i = nodes.size() - 1; i >= 0; i--) {
        int u = nodes[i], p = parent[u];
        if (p != -1) {
            answer += (distance[u] - distance[p]) * subtree[u] * (total - subtree[u]);
            subtree[p] += subtree[u];
        }
    }
    cout << answer << '\n';
    for (int u : nodes) weight[u] = 0;
}
}

```

```
import sys
```

```
def main():
```

```

t = sys.stdin.buffer.read().split()
if not t:
    return
z = 0
n = int(t[z])
q = int(t[z + 1])
z += 2
g = [[] for _ in range(n)]
for _ in range(n - 1):
    u = int(t[z]) - 1
    v = int(t[z + 1]) - 1
    w = int(t[z + 2])
    z += 3
    g[u].append((v, w))
    g[v].append((u, w))
log = n.bit_length()
up = [[0] * n for _ in range(log)]
tin = [0] * n
tout = [0] * n
dist = [0] * n
timer = 0
stack = [(0, 0, 0)]
while stack:
    u, p, state = stack.pop()
    if not state:
        tin[u] = timer
        timer += 1
        up[0][u] = p
        for j in range(1, log):
            up[j][u] = up[j - 1][up[j - 1][u]]
        stack.append((u, p, 1))
        for v, w in reversed(g[u]):
            if v != p:
                dist[v] = dist[u] + w
                stack.append((v, u, 0))
    else:
        tout[u] = timer - 1

def anc(u, v):
    return tin[u] <= tin[v] <= tout[u]

def lca(u, v):
    if anc(u, v):
        return u
    if anc(v, u):
        return v
    for j in range(log - 1, -1, -1):
        if not anc(up[j][u], v):
            u = up[j][u]
    return up[0][u]

```

```

weight = [0] * n
sub = [0] * n
parent = [-1] * n
out = []
for _ in range(q):
    k = int(t[z])
    z += 1
    nodes = []
    total = 0
    for _ in range(k):
        u = int(t[z]) - 1
        c = int(t[z + 1])
        z += 2
        nodes.append(u)
        weight[u] = c
        total += c
    nodes.sort(key=tin.__getitem__)
    nodes += [lca(nodes[i - 1], nodes[i]) for i in range(1, k)]
    nodes = sorted(set(nodes), key=tin.__getitem__)
    st = []
    for u in nodes:
        while st and not anc(st[-1], u):
            st.pop()
        parent[u] = st[-1] if st else -1
        st.append(u)
        sub[u] = weight[u]
    answer = 0
    for u in reversed(nodes):
        p = parent[u]
        if p != -1:
            answer += (dist[u] - dist[p]) * sub[u] * (total - sub[u])
            sub[p] += sub[u]
    out.append(str(answer))
    for u in nodes:
        weight[u] = 0
print("\n".join(out))

if __name__ == "__main__":
    main()

```

6 F. Subtree Mode Profile

Process bottom-up, retaining the largest child frequency map and merging all smaller maps into it. While increasing a color count, reset the number of modes to one on a new maximum or increment it on a tie.

Proof. The merged maps partition the descendants, and adding the current color completes the subtree. The update rule maintains exactly the largest frequency and the number of colors equal to it. Smaller-to-larger merging does not alter counts, only their storage.

Each entry moves $O(\log n)$ times, giving expected $O(n \log n)$ time and $O(n)$ memory. Test all colors equal, all distinct, and several tied modes.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n)) return 0;
    vector<int> color(n);
    for (int& x : color) cin >> x;
    vector<vector<int>> g(n);
    for (int i = 1, u, v; i < n; i++) {
        cin >> u >> v;
        --u;
        --v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<int> parent(n, -1), order{0};
    for (int i = 0; i < (int)order.size(); i++) {
        int u = order[i];
        for (int v : g[u])
            if (v != parent[u]) parent[v] = u, order.push_back(v);
    }
    vector<unordered_map<int, int>*> maps(n);
    vector<int> best(n), ways(n);
    for (int at = n - 1; at >= 0; at--) {
        int u = order[at], base = -1;
        for (int v : g[u])
            if (parent[v] == u && (base < 0 || maps[v]->size() > maps[base]->size()))
                base = v;
        if (base < 0)
            maps[u] = new unordered_map<int, int>();
        else
```

```

    maps[u] = maps[base], best[u] = best[base], ways[u] = ways[base];
    auto add = [&](int c, int amount) {
        int value = ((*maps[u])[c] += amount);
        if (value > best[u])
            best[u] = value, ways[u] = 1;
        else if (value == best[u])
            ways[u]++;
    };
    for (int v : g[u])
        if (parent[v] == u && v != base) {
            for (auto [c, x] : *maps[v]) add(c, x);
            delete maps[v];
        }
    add(color[u], 1);
}
for (int u = 0; u < n; u++) cout << best[u] << ' ' << ways[u] << '\n';
delete maps[0];
}

```

```

import sys

def main():
    d = list(map(int, sys.stdin.buffer.read().split()))
    if not d:
        return
    n = d[0]
    colors = d[1 : n + 1]
    z = n + 1
    g = [[] for _ in range(n)]
    for _ in range(n - 1):
        u = d[z] - 1
        v = d[z + 1] - 1
        z += 2
        g[u].append(v)
        g[v].append(u)
    parent = [-1] * n
    order = [0]
    for u in order:
        for v in g[u]:
            if v != parent[u]:
                parent[v] = u
                order.append(v)
    maps = [None] * n
    best = [0] * n
    ways = [0] * n
    for u in reversed(order):
        children = [v for v in g[u] if parent[v] == u]
        base = max(children, key=lambda v: len(maps[v]), default=-1)
        if base < 0:
            current = {}

```

```

        maximum = count = 0
    else:
        current = maps[base]
        maximum = best[base]
        count = ways[base]

    def add(color, amount):
        nonlocal maximum, count
        value = current.get(color, 0) + amount
        current[color] = value
        if value > maximum:
            maximum = value
            count = 1
        elif value == maximum:
            count += 1

    for v in children:
        if v != base:
            for color, amount in maps[v].items():
                add(color, amount)
            maps[v].clear()
    add(colors[u], 1)
    maps[u] = current
    best[u] = maximum
    ways[u] = count
    print("\n".join(f"{best[u]} {ways[u]}" for u in range(n)))

if __name__ == "__main__":
    main()

```

7 Stress testing and carry-forward

All six random generators use copied sequences, fresh BFS components, explicit path products, all-pairs distances, or complete subtree counters. The important master+ habit is to validate both the compressed structure and the ordering in which its summaries are combined.